An Autonomous, polars-Native Engine for Data Cleaning and Multi-Domain Statistical Analysis

Nicolo Angileri - Founder & Lead Researcher, Mediterranean Quantitative Finance Society (MQFS)

Abstract

We present auto_cleaner, an autonomous engine that ingests messy tabular data (CSV, Excel, Parquet, JSON, SQL, FITS, netCDF) and returns a mathematically clean, model-ready dataset together with a comprehensive, reproducible analysis. The system is built end to end on polars for multi-threaded, columnar computation, with DuckDB for SQL ingestion. Two design choices define it. First, auto-specialisation: the engine inspects each dataset's schema and value patterns, infers one or more archetypes (time-series, geospatial, text-heavy, high-dimensional, survey, wide/omics), and routes the analyses that genuinely fit. Second, quantified self-scrutiny: every distribution-altering cleaning step is measured (Kolmogorov-Smirnov distance, mean shift in pre-cleaning standard deviations, share of cells changed) and material distortions are escalated to an executive summary with a human review checklist. On the public auto-mpg benchmark the engine reaches a composite data-quality score of 91.9/100, reduces memory by 44.4% through provably safe down-casting, recovers the expected multicollinearity structure (VIF up to 20), and normalises a skewed feature by Box-Cox (skewness +1.05 to +0.02). The engine is further validated, unmodified, on 10 public real-world datasets across nine domains - 3,038,121 rows from finance to climate to AI - without a single failure. The value of such a tool lies not in replacing human judgement but in compressing the first, repetitive 60-80% of any data project into a single, transparent, reproducible step.

1. Introduction

Every empirical study begins with the same unglamorous work: ingesting messy files, fixing types, handling missing values and outliers, and performing enough exploratory analysis to understand the data before any modelling. This phase is repetitive, error-prone, and rarely reproducible. auto_cleaner automates it as a single command, while remaining explicit about every decision it makes. The engine is deliberately framed as an accelerator and diagnostic tool: it produces baselines and well-structured evidence for a human to validate, not autonomous conclusions.

2. System design

The engine is a functional pipeline. Each stage is a pure function of the form DataFrame -> (DataFrame, Report), and the pipeline is their composition: ingest -> validate -> standardise -> impute -> outliers -> downcast -> profile -> specialise -> analysis -> report. State lives only in the data flowing through and in lightweight report accumulators, which makes stages independently testable and reusable. Optional backends (scikit-learn, statsmodels, astropy, sentence-transformers) are imported lazily, so a missing dependency degrades a single feature rather than breaking a run.

Ingestion is hardened for real files: malformed delimited data is recovered through a graceful-degradation ladder (strict parse -> all-string fallback -> tolerant with ragged-line truncation -> quote-free last resort) in which every truncated or dropped line is counted and surfaced as a warning, never discarded silently; encodings are sniffed down to BOM-less UTF-16/32; Excel workbooks are read via the calamine engine with explicit worksheet accounting. The cleaning stage performs safe integer/float down-casting, context-aware imputation (forward-fill for ordered series, median for skewed and mean for symmetric distributions), and outlier treatment by interquartile range, z-score and Isolation Forest. Three execution profiles (fast, standard, full) trade analytic depth for latency.

3. Quantified cleaning impact

Imputation and outlier treatment are interventions on the data: they change the very distributions the analysis then reports. auto_cleaner therefore measures its own footprint. For each numeric column, with empirical distribution functions F_pre and F_post before and after a step, it computes

$$D = \sup_x \left| \hat{F}_{\text{pre}}(x) - \hat{F}_{\text{post}}(x) \right|, \qquad \delta = \frac{|\mu_{\text{post}} - \mu_{\text{pre}}|}{\sigma_{\text{pre}}}, \qquad s = \frac{\#\{\text{cells changed}\}}{n}$$

where D is the two-sample Kolmogorov-Smirnov distance, delta the mean shift in units of the pre-cleaning standard deviation, and s the share of cells the step modified. A conservative verdict is attached to every column:

$$\text{material}: \; s \geq 0.10 \;\vee\; \delta \geq 0.10 \;\vee\; D \geq 0.15; \qquad \text{negligible}: \; s < 0.01 \;\wedge\; \delta < 0.02 \;\wedge\; D < 0.05$$

with everything in between labelled minor. Material verdicts are escalated three times: as a data-health warning, as a row in the per-step Cleaning Impact table, and as a headline in the report's executive summary, which opens every report with the findings ranked worst-first and a review checklist of the decisions a human must sign off. The design goal is that an analyst can trust a cleaned column because the evidence that the cleaning was benign travels with it.

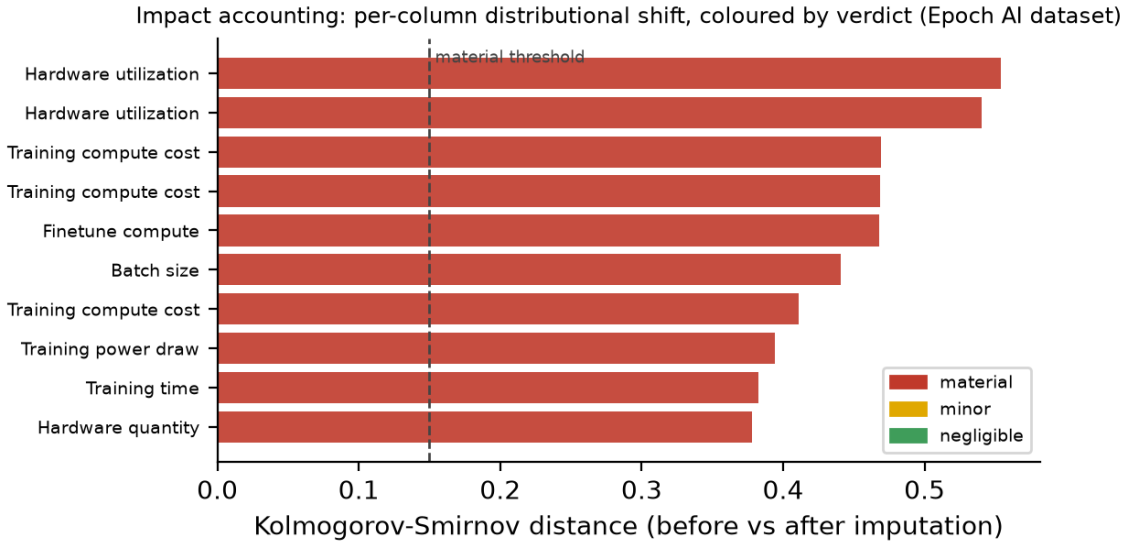

Figure 1. Impact accounting in action: a mostly-missing real-data column before and after imputation. The engine measures the distributional shift and flags it as material, rather than blessing it silently.

4. Capabilities

Beyond cleaning, the engine computes a full statistical profile (skewness, kurtosis, correlation and covariance, missingness) and a battery of advanced analyses, each gated by the detected archetype: normality testing (Shapiro-Wilk, D'Agostino, Jarque-Bera, Anderson-Darling); distribution fitting with AIC/BIC selection; robust location estimators; rank, partial and categorical association measures with effect sizes; multicollinearity diagnostics (VIF, PCA); multivariate analysis (Mahalanobis distance, clustering, MANOVA, UMAP); classical natural-language processing; Bayesian comparison; survival and survey methodology; functional data analysis; and time-series diagnostics with ARIMA/Holt-Winters forecasting. A baseline modelling layer reports cross-validated benchmarks with permutation and SHAP importances, and an opt-in layer adds Optuna hyper-parameter tuning and, only when a treatment and outcome are explicitly declared, A/B testing and an observational causal estimate by propensity inverse-probability weighting. Outputs include an interactive HTML report, Markdown, a per-dataset PDF that opens with the executive summary, a machine-readable results manifest, standalone figures, and a serialised model with a batch-scoring command.

5. Empirical demonstration

The table below reports genuine results from a single run on the public auto-mpg dataset (406 observations, 9 source features), unedited. The engine recovers the textbook structure of the data: the engine-size variables are severely collinear (Cylinders-Displacement, r = 0.95), Horsepower is right-skewed and is normalised by a Box-Cox transform, and a tree ensemble provides the strongest cross-validated baseline. The imputation footprint on this run was measured at Miles_per_Gallon: 8 cells (2.0%), KS 0.010 (minor) - the cleaning demonstrably did not distort the data it reports on. On an independent daily climate series the engine instead auto-detected a time-series archetype and produced twelve-step forecasts with 95% prediction intervals; a drift comparison against a perturbed copy isolated the single shifted feature (population stability index 0.93, major drift). The same command therefore yields materially different, appropriate analyses on different data, with no manual configuration.

Metric	Value
Observations x source features	406 x 9
Composite data-quality score	91.9 / 100
In-memory reduction (safe down-casting)	44.4 %
Best baseline model (5-fold CV R2)	Random Forest (R2 = 0.41)
Strongest multicollinearity (VIF)	Displacement (VIF = 20.1)
Top predictor (mutual information)	Displacement
Box-Cox normalisation (skewness)	Horsepower: +1.05 -> +0.02

Largest imputation footprint	Miles_per_Gallon: 8 cells (2.0%), KS 0.0.

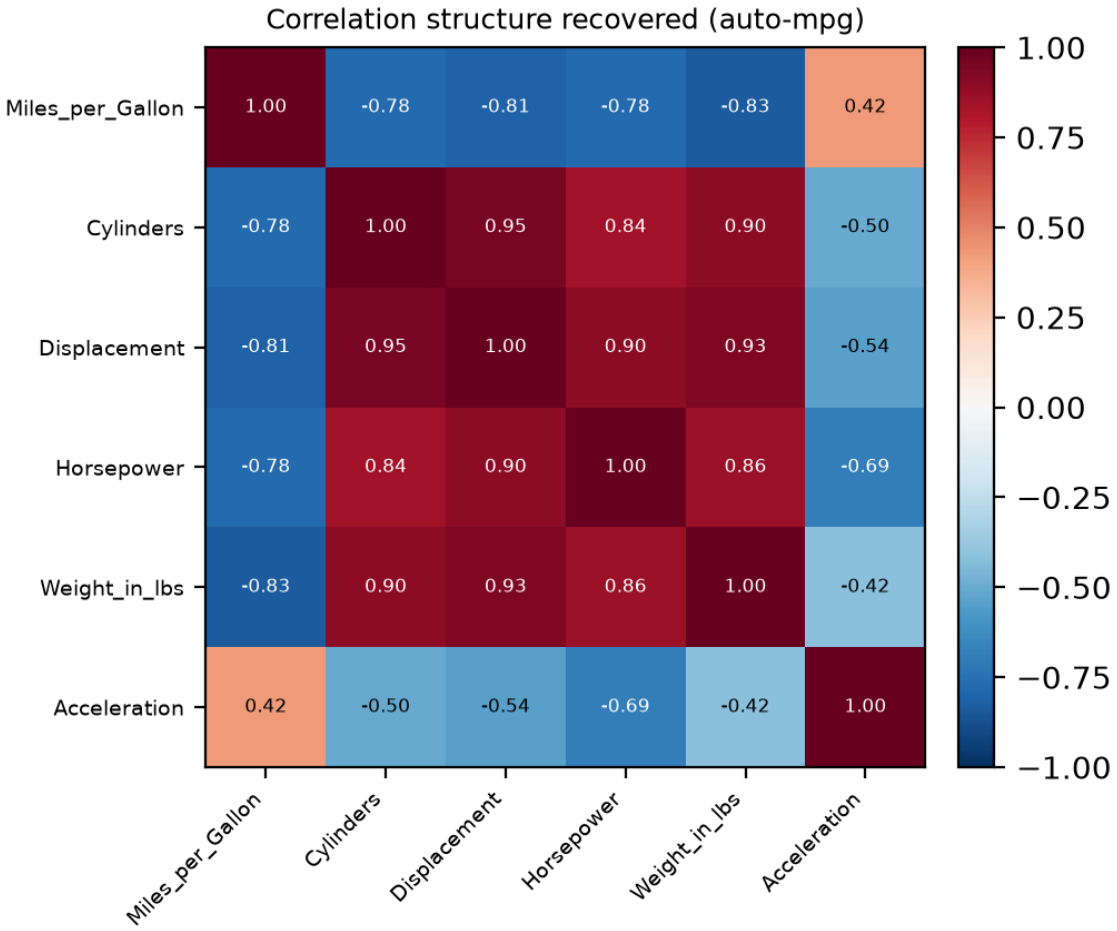

Figure 2. Correlation structure the engine recovers on auto-mpg - the severe engine-size collinearity (Cylinders/Displacement/Weight) that drives the VIF warnings.

6. Validation on ten real-world datasets

To validate the engine beyond curated demos, we ran it unmodified over 10 public real-world datasets spanning finance (credit risk and FX markets), the automotive domain, artificial intelligence, climate science, census microdata, cardiology, field biology, food chemistry and urban mobility - 3,038,121 rows in total, every file taken from its primary source with no preprocessing (all sources are cited in the References). Every dataset was ingested, cleaned and profiled without a crash or a silent failure. The runs exercised the hardening paths on genuine dirt: the NASA GISTEMP export opens with a caption line before the header (auto-detected and skipped, with its '***' missing-value markers mapped to nulls); the UCI credit-risk data arrives as a legacy .xls workbook (read through the calamine engine, worksheet accounted for); the UCI census and cardiology files carry no header row (detected, synthetic names assigned); and the ECB FX series embeds sparse SDMX metadata columns.

Domain	Rows x cols	Quality	Time
Finance (credit risk)	30,001 x 26	74/100	8.9s
Finance (FX markets)	7,102 x 33	77/100	0.2s
Automotive	406 x 10	92/100	8.5s
Artificial intelligence	1,033 x 48	68/100	0.4s
Climate	147 x 20	74/100	0.1s
Social / census	32,562 x 16	89/100	2.1s
Medicine (cardiology)	303 x 15	97/100	0.1s
Biology (field data)	344 x 8	97/100	0.8s
Food chemistry	1,599 x 13	83/100	4.8s
Transport (urban mobility)	2,964,624 x 20	73/100	3.8s

Scale: the January-2024 NYC yellow-taxi file (2,964,624 rows x 20 columns, Parquet) was cleaned and profiled in 3.8 seconds with the fast profile and streaming ingestion. A measured scale probe over concatenated copies sustains over 450,000 rows/second up to

11,858,496 rows (26 s, 3.0 GB peak RSS) - the practical single-machine envelope before the documented Spark/Ray hand-off. The impact accounting also proved itself on real data: on the artificial intelligence dataset, imputing the mostly-missing column 'Hardware utilization (HFU)' shifted its distribution by a Kolmogorov-Smirnov distance of 0.55 (98% of cells changed). The engine flagged its own intervention as material, escalated it to the executive summary, and left the decision to the human - which is precisely the intended behaviour: an automated cleaner must not silently bless a column it has materially reshaped.

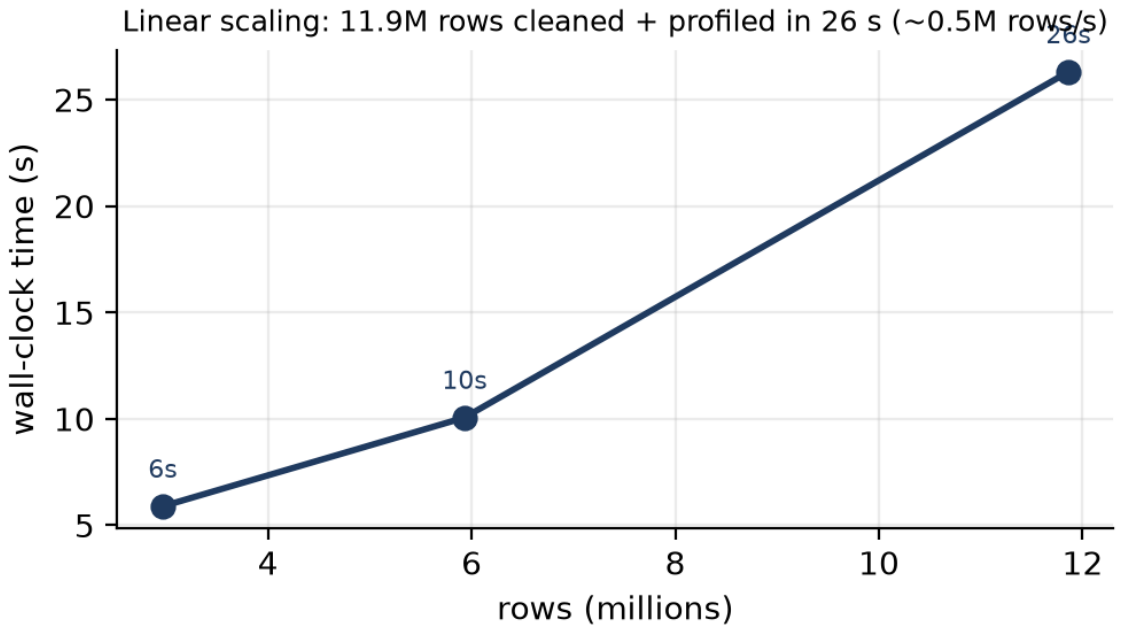

Figure 3. Measured throughput: wall-clock time scales linearly with row count, sustaining ~0.5M rows/s to 11.9M rows (fast profile, streaming ingestion).

7. Scope and limitations

We are deliberate about the engine's ceiling. It is a single-machine system: data larger than memory is handled by streaming and DuckDB, but cluster-scale, distributed computation is a different engine (Spark/Ray) and is out of scope, as is real-time streaming. Its modelling and inference outputs are baselines and exploratory diagnostics, not causal proof or production models; a statistical-hygiene guard explicitly warns about multiple comparisons, and the causal layer ships with prominent assumptions (ignorability, positivity). The engine is domain-aware through heuristics, not a domain expert: it routes by archetype but does not apply field-specific methodology such as survey weighting design or instrument calibration. This clarity is the point: the tool excels at turning raw, messy, single-machine-scale data into a clean dataset and a rigorous first analysis, and defers the domain-specific and production steps to a human.

8. Conclusion

auto_cleaner demonstrates that a large fraction of applied data work - cleaning, exploratory and confirmatory statistics, baseline modelling, and reporting - can be automated rigorously and transparently across domains, on a single machine, while remaining honest about what requires human judgement, and while measuring the footprint of its own interventions. The result is an accelerator that compresses hours of preparatory work into one reproducible command.

References

[1] R. Vink et al. Polars: Lightning-fast DataFrame library for Rust and Python. Software.
[2] F. Pedregosa et al. Scikit-learn: Machine Learning in Python. JMLR 12:2825-2830, 2011.
[3] S. Seabold, J. Perktold. Statsmodels: Econometric and Statistical Modeling with Python. Proc. 9th Python in Science Conf., 2010.
[4] M. Raasveldt, H. Muhleisen. DuckDB: An Embeddable Analytical Database. Proc. SIGMOD, 2019.
[5] S. Lundberg, S.-I. Lee. A Unified Approach to Interpreting Model Predictions. NeurIPS, 2017.
[6] T. Akiba et al. Optuna: A Next-generation Hyperparameter Optimization Framework. Proc. KDD, 2019.
[7] F. J. Massey. The Kolmogorov-Smirnov Test for Goodness of Fit. JASA 46(253):68-78, 1951.
[8] I-Cheng Yeh, C. Lien. Default of Credit Card Clients. UCI Machine Learning Repository, 2016.
[9] European Central Bank. US dollar/Euro reference exchange rate, daily (EXR.D.USD.EUR.SP00.A). ECB Data Portal, retrieved 2026.
[10] R. Quinlan. Auto MPG. UCI Machine Learning Repository, 1993 (via vega-datasets).
[11] Epoch AI. Notable AI Models database. epoch.ai/data, retrieved 2026.
[12] GISTEMP Team. GISS Surface Temperature Analysis v4. NASA GISS, retrieved 2026.

[13] B. Becker, R. Kohavi. Adult (Census Income). UCI Machine Learning Repository, 1996.
[14] A. Janosi et al. Heart Disease (Cleveland). UCI Machine Learning Repository, 1988.
[15] K. Gorman, A. Horst, A. Hill. Palmer Archipelago Penguins. PLoS ONE 9(3), 2014.
[16] P. Cortez et al. Wine Quality. UCI Machine Learning Repository / Decision Support Systems 47(4), 2009.
[17] NYC Taxi & Limousine Commission. Yellow Taxi Trip Records, January 2024. nyc.gov/tlc.